Aegis Clinical: An Offline Edge-Deployed Voice-to-Prescription Assistant Using Whisper ASR, Phonetic Medicine Normalization and Quantized Local Language Models

Student Name 1
Reg. No. XXXXXXXX
Dept. of Electronics & Communication Engg.
Name of the Institution
City, Country
email@institution.edu

Student Name 2
Reg. No. XXXXXXXX
Dept. of Electronics & Communication Engg.
Name of the Institution
City, Country
email@institution.edu

Student Name 3
Reg. No. XXXXXXXX
Dept. of Electronics & Communication Engg.
Name of the Institution
City, Country
email@institution.edu

Faculty Name (Guide)
Assistant Professor
Dept. of Electronics & Communication Engg.
Name of the Institution
City, Country
email@institution.edu

***Abstract*—Handwritten and manually typed prescriptions consume a significant share of outpatient consultation time and are prone to legibility and transcription errors, while cloud-based speech services raise connectivity and patient-privacy concerns. This paper presents Aegis Clinical, an offline-first, clinician-in-the-loop system that converts a doctor's spoken dictation into a structured, editable prescription on a low-cost edge device such as the Raspberry Pi 5. Audio from a microphone, system loopback or an uploaded file is downmixed, resampled to 16 kHz and conditioned using DC-offset removal, a fifth-order Butterworth band-pass filter and RMS normalization. Speech is transcribed by an int8-quantized faster-whisper model biased with a clinical vocabulary prompt. A deterministic pipeline then cleans recurrent ASR distortions, extracts complaints, diagnoses, medications, investigations and advice, and normalizes medicine names against a 467-entry formulary using combined Soundex, medical-phonetic and fuzzy matching. An optional 4-bit GGUF language model (Qwen2.5 1.5B via llama.cpp) refines the structured output, and a rule engine flags missing fields, duplicate ingredients and selected drug–drug and disease–drug conflicts before SQLite storage and PDF generation. On a 50-case clinical transcript benchmark the hybrid extractor with LLM refinement achieved 87.69% medication recall, a 96.0% diagnosis hit rate and a mean refinement latency of 9.55 s per case, demonstrating that useful prescription drafting is feasible fully on-device.**

Keywords—speech recognition, Whisper, edge computing, Raspberry Pi, clinical NLP, e-prescription, phonetic matching, quantized LLM, offline healthcare

I. INTRODUCTION

Clinical documentation is one of the largest contributors to physician workload. In busy outpatient departments, particularly in India and other high-volume settings, a doctor may see dozens of patients per session and must still write or type a prescription for each one. Handwritten prescriptions are a well-known source of legibility and medication errors, while electronic health record (EHR) data entry diverts the clinician's attention away from the patient. Ambient and dictation-based AI scribes have recently been shown to reduce this documentation burden [13], and large language models (LLMs) have demonstrated strong performance on clinical knowledge and summarization tasks [6], [7], [14].

However, most existing speech-to-text and AI-scribe solutions depend on cloud inference. This creates three practical problems: (i) clinics with unreliable internet cannot use them; (ii) sensitive patient audio leaves the premises, complicating privacy and data-governance obligations; and (iii) recurring subscription costs are prohibitive for small practices. In addition, general-purpose automatic speech recognition (ASR) frequently confuses Indian brand names (e.g., “Dolo 650”, “Pan-D”, “Augmentin 625”), dosing abbreviations (“1-0-1”, “BD”, “SOS”) and laboratory terms (“HbA1c”), and Whisper-class models are known to occasionally hallucinate content that was never spoken [15].

Recent advances make a fully local alternative feasible. Robust weakly supervised ASR models such as Whisper [1] and its optimized derivatives [8], [9] can run on CPUs, and post-training quantization techniques [4], [5] allow compact instruction-tuned LLMs [2], [3] to execute on single-board computers. Building on these developments, this work proposes Aegis Clinical, an embedded voice-to-prescription assistant whose entire inference path runs on the edge device. The main contributions are:

- an end-to-end offline pipeline from multi-source audio capture to a reviewable PDF prescription, deployable on a Raspberry Pi 5 or a commodity PC;
- a lightweight DSP front-end and clinically prompted, int8-quantized Whisper ASR suited to CPU-only edge hardware;
- a hybrid extraction scheme combining deterministic rules, a 467-key medicine formulary and a weighted Soundex/medical-phonetic/fuzzy matcher, optionally refined by a 4-bit local LLM with graceful fallback;
- a clinical rule layer and mandatory clinician confirmation that keep the doctor in control of every prescription; and
- a 50-case benchmark quantifying medication recall, diagnosis detection and latency.

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the methodology and system architecture, Section IV presents results and discussion, and Section V concludes the paper.

II. RELATED WORK

End-to-end neural ASR has progressed from self-supervised encoders such as wav2vec 2.0 [11] and convolution-augmented Transformers such as Conformer [10] to Whisper [1], which was trained on 680,000 hours of weakly labeled multilingual audio and generalizes robustly without

fine-tuning. Distil-Whisper [8] reduces model size through knowledge distillation, and WhisperX [9] adds voice-activity-based segmentation and word-level timestamps for long-form audio. Nevertheless, Koenecke et al. [15] showed that Whisper can hallucinate phrases, which is a serious risk in medical documentation and motivates downstream validation.

In clinical NLP, domain-specific pre-training such as PubMedBERT [16] improved biomedical entity recognition, and the 2018 n2c2 shared task [17] established benchmarks for medication and adverse-drug-event extraction from EHR text. Agrawal et al. [12] demonstrated that LLMs are effective few-shot clinical information extractors, Singhal et al. [6] showed that LLMs encode substantial clinical knowledge, and Van Veen et al. [14] reported that adapted LLMs can match or outperform experts at clinical text summarization.

For edge deployment, Chen and Ran [18] surveyed deep learning on edge devices, highlighting latency, bandwidth and privacy benefits. Quantization approaches such as LLM.int8() [4] and GPTQ [5] reduce memory by 2–4× with modest accuracy loss, enabling 1–3 B parameter models such as Qwen2.5 [2] and Llama-family models [3] to run on CPUs. Unlike prior cloud-based scribes, the proposed system combines these components into a single offline embedded workflow with deterministic safety checks. Table VII at the end of Section IV compares the proposed system with representative approaches.

III. METHODOLOGY

A. Design Requirements

The system was designed around five requirements derived from outpatient practice: (R1) offline operation, so that no audio or patient data leaves the clinic device; (R2) low-cost CPU-only hardware with at most 8 GB of RAM; (R3) robustness to Indian brand names, abbreviated dosing schedules and inexpensive microphones; (R4) graceful degradation, so that failure of any optional component still yields a usable draft; and (R5) clinician control, so that no prescription is finalized without explicit confirmation. Each component described below is chosen to satisfy one or more of these requirements.

B. System Architecture

Fig. 1 shows the block diagram of the proposed system. The application is implemented in Python as a Flask web server with an HTML/JavaScript front end that can be accessed from any browser on the local network. Each stage is described below.

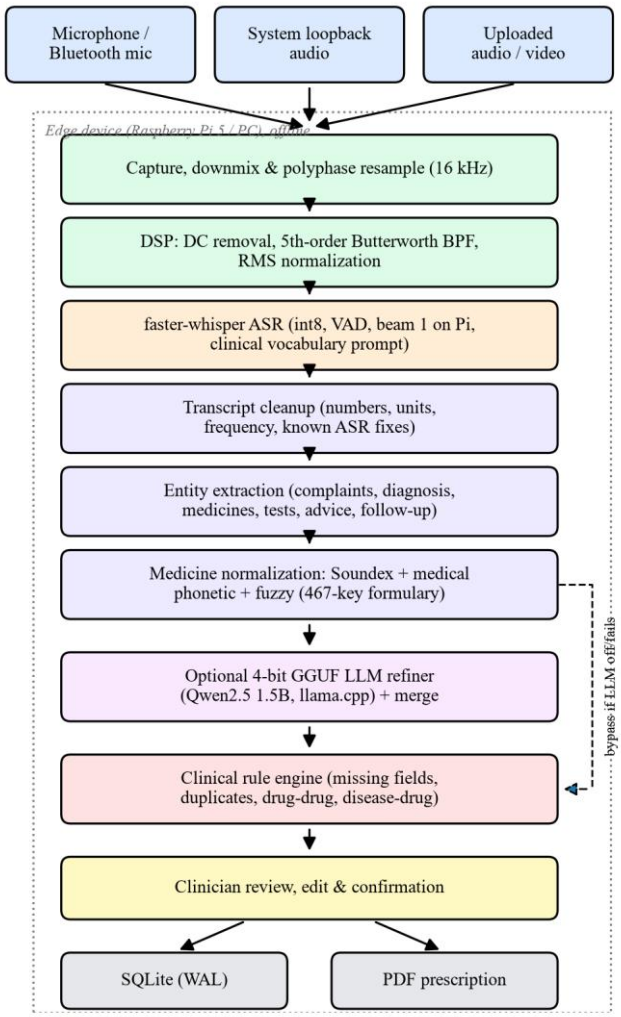


Fig. 1. Block diagram of the proposed Aegis Clinical voice-to-prescription system.

C. Audio Acquisition and Resampling

Three sources are supported: a USB or Bluetooth microphone captured through PortAudio, system loopback audio, and uploaded audio/video files decoded by ffmpeg. The capture module tries a priority list of sample-rate and channel configurations (16 kHz mono first, then the device's native rate, 44.1 kHz and 48 kHz) so that it works with heterogeneous hardware. Multi-channel audio with C channels is downmixed by averaging,

$$x[n] = (1/C)\,\Sigma c\; xc[n] \quad (1)$$

and, when the capture rate fc differs from 16 kHz, polyphase resampling with up/down factors L = 16000/gcd(16000, fc) and M = fc/gcd(16000, fc) is applied. The signal is clipped to [−1, 1] and stored as 16-bit PCM.

D. Signal Pre-processing

To improve ASR robustness to low-cost microphones, the waveform x of length N is conditioned by DC-offset removal, band-pass filtering and RMS normalization:

$$\tilde{x}[n] = x[n] - (1/N)\,\Sigma i\; x[i] \quad (2)$$

$$y = SOSfilt(Butter5(100\ Hz, fH), \tilde{x}) \quad (3)$$

$$z[n] = clip(\, y[n] \cdot 0.08 / rms(y), -1, 1\,) \quad (4)$$

where fH = min(8000, fs/2 − 1). The 100 Hz lower cut-off suppresses mains hum and handling noise, and the fifth-order

Butterworth filter is realized in second-order sections for numerical stability. If any DSP step fails, the original audio is passed through unchanged.

E. Clinical Speech Recognition

Transcription uses faster-whisper (a CTranslate2 re-implementation of Whisper [1]) with base, small or medium checkpoints stored locally, so no network access is required. The model runs on the CPU with int8 weights, temperature 0 for deterministic decoding, built-in voice activity detection, and a beam size of 1 on the Raspberry Pi (5 on a PC). An initial prompt containing common brand and generic drug names, units, tests and dosing abbreviations biases the decoder towards clinical vocabulary:

$$\hat{w} = argmax\, \Sigma t\ log\ P(wt \mid w<t, p, a) \quad (5)$$

where p is the clinical prompt and a the audio. The original OpenAI Whisper implementation is retained as a fallback backend.

F. Transcript Cleanup and Entity Extraction

A deterministic cleaner applies, in order, spoken-number normalization, unit normalization ("five hundred milligram" → "500 mg"), frequency normalization ("twice a day" → "BD"), a table of known ASR corrections and punctuation injection. The extractor then produces the tuple E = (C, D, M, I, A, F, N) of complaints, diagnoses, medications, investigations, advice, follow-up and notes. Symptom and disease lexicons are scanned longest-match-first; medications are found using a cached formulary regex, name–dose patterns and prescription-verb patterns. Clause isolation keeps dose, frequency, duration, route and instructions attached to the correct drug, and negation phrases such as "stop" or "do not take" suppress candidates. Frequency patterns such as 1-0-1, 1-1-1 and "as needed" are mapped to BD, TID and SOS.

G. Phonetic and Fuzzy Medicine Normalization

Because ASR often produces near-homophones of drug names, each candidate token t is matched against every formulary key v (467 entries with brand, generic, dose, route and category metadata). With r the SequenceMatcher similarity, δs the Soundex agreement and δp the agreement of a custom medical-phonetic key, the score is

$$s(t,v) = min(1,\ r + 0.22\delta s\delta p + 0.12(\delta s \oplus \delta p)) \quad (6)$$

A prefix match with $|t| \geq 4$ raises the score to at least 0.88. The best match is accepted at $s \geq 0.65$ and flagged for review when $s < 0.80$. A blacklist of common non-drug words prevents false positives, and recognized pharmaceutical suffixes admit plausible unseen names at confidence 0.70.

H. Optional Local LLM Refinement

When enabled, a 4-bit GGUF model (Qwen2.5 1.5B Instruct by default; Llama 3.2 1B Medical and Qwen2.5 3B Medical are selectable) runs through llama.cpp on four CPU threads with a 2048-token context and temperature 0. The prompt contains the deterministic draft and the last 1200 transcript characters. The JSON response is parsed with progressive repair, keys are normalized, and results are merged with the deterministic draft; any LLM medicine absent from the formulary is marked needs_review. A re-entrant lock serializes model access, and any failure returns the deterministic draft unchanged, so the LLM can only improve, never block, output. This control flow is shown in Fig. 2, and the selectable models are listed in Table I.

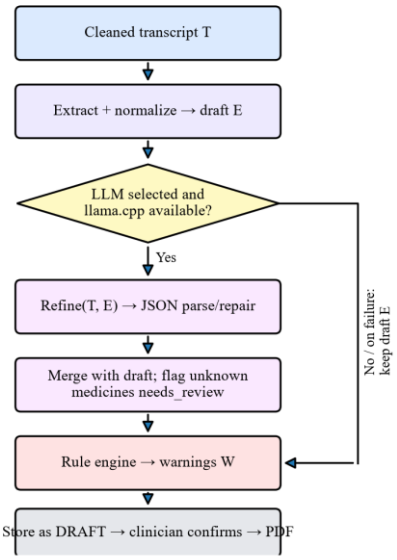


Fig. 2. Draft generation flow with optional LLM refinement and fallback.

TABLE I. SELECTABLE LOCAL LLM CONFIGURATIONS

Model (GGUF)	Params	Approx. RAM	Role
Qwen2.5 Instruct	1.5 B	~1.1 GB	Default refiner
Llama 3.2 Medical	1 B	~0.8 GB	Low-memory option
Qwen2.5 Medical	3 B	~2.2 GB	Higher-capacity option

I. Clinical Rule Engine, Storage and Output

The rule engine reports empty medication lists and missing dose or frequency, and detects duplicate paracetamol products, multiple NSAIDs, NSAID with antiplatelet/anticoagulant, macrolide with statin, fluoroquinolone with steroid or antacid, and disease–drug conflicts (beta-blockers in asthma/COPD, NSAIDs without PPI in gastritis, steroids in diabetes). Every generated prescription is stored as a draft in SQLite (WAL mode); only an explicit clinician confirmation changes its status, after which a PDF is rendered with ReportLab. The implemented checks are summarized in Table II.

TABLE II. CLINICAL RULE ENGINE CHECKS

Type	Condition detected	Level
Completeness	No medicines; missing dose/frequency	Info/Warning
Duplication	Multiple paracetamol products; multiple NSAIDs	Warning
Drug–drug	NSAID + antiplatelet/anticoagulant	Warning
Drug–drug	Macrolide + statin	Warning
Drug–drug	Fluoroquinolone + steroid / antacid	Warning
Disease–drug	Beta-blocker in asthma/COPD	Warning
Disease–drug	NSAID without PPI in GI complaints	Info
Disease–drug	Systemic steroid in diabetes	Warning

J. Edge Deployment

The same code base runs on a Windows/Linux PC or a 64-bit Raspberry Pi OS device. A Raspberry Pi is detected from /proc/device-tree/model (or forced with an environment variable), and the edge profile then lowers the Whisper beam size to 1 and disables loopback capture. A one-shot setup script installs ffmpeg, PortAudio and OpenBLAS, creates a virtual environment, builds llama-cpp-python with BLAS and native ARM optimizations, and registers a systemd service so the assistant starts at boot. Whisper checkpoints, GGUF

models and Python wheels are pre-downloaded, allowing installation and operation with no internet connection. A helper script pairs a Bluetooth microphone, and clinicians access the interface from a phone or laptop browser on the clinic network. Table III summarizes the implementation stack.

TABLE III. HARDWARE AND SOFTWARE STACK

Layer	Technology	Purpose
Hardware	Raspberry Pi 5 / x86 PC, USB/BT mic	Edge host
Web/UI	Flask, HTML, CSS, JS	Local UI and REST API
Capture	sounddevice, soundcard	Mic & loopback
DSP	NumPy, SciPy	Filter, resample, normalize
ASR	faster-whisper (int8)	Offline transcription
NLP	Regex, lexicons, Soundex	Extraction, normalization
LLM	llama-cpp-python, GGUF Q4	Structured refinement
Storage/Out	SQLite, ReportLab	Records and PDF

IV. RESULTS AND DISCUSSION

A. Experimental Setup

The extraction and refinement stages were evaluated on a benchmark of 50 synthetic outpatient consultation transcripts spanning 16 specialties, including gastroenterology, cardiology, endocrinology, infectious disease, orthopedics, dermatology, pediatrics and neurology, each annotated with the expected medicines and diagnoses. For every case the transcript was passed through extraction, normalization and Qwen2.5 1.5B refinement. Medication recall for case i is

$$Ri = |Gi \cap \dot{G}i| / |Gi| \quad (7)$$

where Gi and Ġi are the expected and predicted medicine sets; a diagnosis is counted as a hit if any expected diagnosis term is recovered. Wall-clock latency of the extraction-plus-refinement block was also recorded.

B. Overall Performance

TABLE IV. BENCHMARK SUMMARY (50 CASES)

Metric	Value
Medication recall	87.69 %
Diagnosis hit rate	96.0 %
Mean latency per case	9.55 s
Median latency	9.52 s
Min / Max latency	5.74 / 17.64 s
Cases with 100 % med. recall	40 / 50

As shown in Table IV, the hybrid pipeline recovered 87.69 % of expected medicines and identified the correct diagnosis in 48 of 50 consultations. Because the deterministic draft is merged with the LLM output rather than replaced by it, medicines recovered by the formulary and phonetic matcher are retained even when the LLM omits them.

C. Category-wise Analysis

TABLE V. CATEGORY-WISE RESULTS

Category	Cases	Med. recall (%)	Diag. hit (%)	Latency (s)
Gastroenterology	5	93.3	80	9.01
Cardiology	5	83.3	100	9.22
Endocrinology	5	93.3	100	8.97
Infectious Disease	5	80.0	100	8.48
Orthopedics	5	66.7	100	11.15
Dermatology	5	100.0	100	9.33
Pediatrics	5	93.3	100	8.56
Neurology	3	77.8	100	9.59
Others (8 spec.)	12	94.4	92	10.45

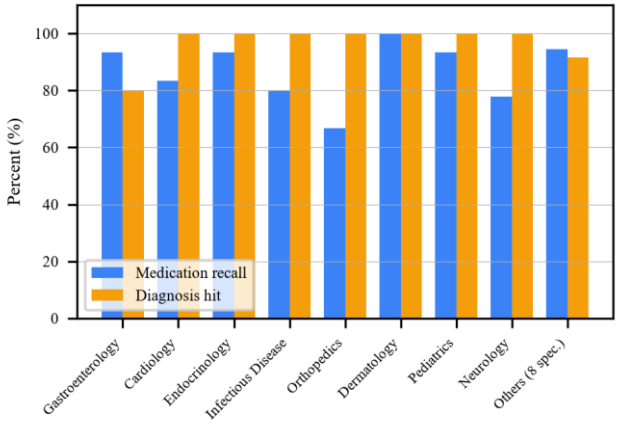


Fig. 3. Category-wise medication recall and diagnosis hit rate.

Table V and Fig. 3 show that dermatology, pediatrics, endocrinology and gastroenterology achieve 93–100 % medication recall, while orthopedics (66.7 %) is the weakest group: one orthopedic case (case_26) and one infectious-disease case (case_24) had zero medication recall. Combination regimens are also difficult; the pulmonology asthma case (case_03), involving combination inhalers where one spoken brand maps to several generics, reached only 33.3 % recall. Diagnosis was missed only in one gastroenterology case (case_10) and one psychiatry case (case_39). These failures indicate that the formulary and lexicons should be expanded for analgesic, antibiotic and inhaler brands.

D. Latency Analysis

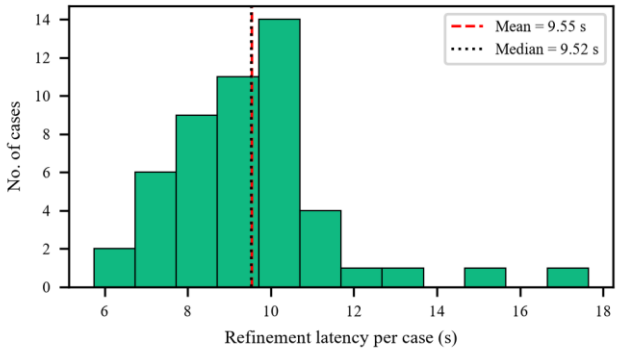


Fig. 4. Distribution of per-case extraction and LLM refinement latency.

Fig. 4 shows that latency ranges from 5.74 s to 17.64 s with a median of 9.52 s. Latency is dominated by autoregressive LLM token generation and depends on the length of the generated output. The deterministic stages consist only of regular expressions and dictionary look-ups, so disabling the LLM gives a much faster draft, while enabling it yields a richer draft within about ten seconds on the test machine. Latency on the Raspberry Pi 5 is expected to be higher and should be profiled separately.

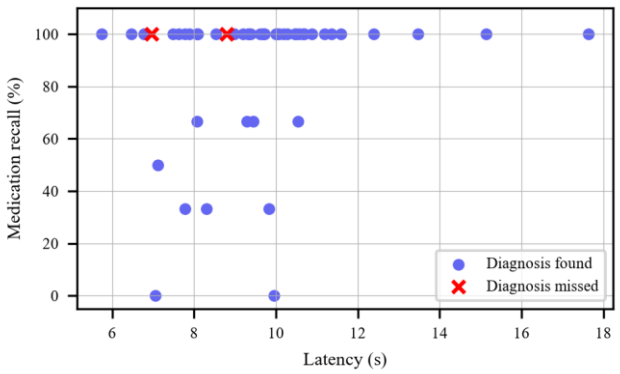


Fig. 5. Per-case medication recall versus latency (diagnosis misses marked ×).

Fig. 5 plots recall against latency for each case. Low-recall cases occur across the latency range rather than only in the slowest cases, which suggests that the errors arise from lexicon coverage and name matching rather than from the LLM running out of output tokens.

E. Worked Example

To illustrate the deterministic stage, the following dictation (written as Whisper typically outputs it) was processed by the cleaner, extractor, normalizer and rule engine without the LLM: "Patient complains of fever and sore throat for three days. Diagnosis is acute pharyngitis. Start Augmentin 625 1-0-1 for five days after food. Dolo 650 SOS for fever. Ibuprofen 400 mg twice a day. Advise warm saline gargles. Review after five days." The actual output is given in Table VI.

TABLE VI. DETERMINISTIC OUTPUT FOR THE SAMPLE DICTATION

Field	Extracted value
Complaints	Sore throat, fever
Diagnosis	Acute pharyngitis
Medicine 1	Amoxicillin (from Augmentin), BD, 5 days, after food; conf. 0.67, needs review
Medicine 2	Paracetamol (from Dolo), SOS; exact match
Medicine 3	Ibuprofen 400 mg, BD; exact match
Medicine 4	Epinephrine (false positive); conf. 0.75, needs review
Advice	Warm saline gargles
Follow-up	5 days
Rule alerts	3 missing-strength notes; missing frequency for Epinephrine

The brand names Augmentin and Dolo were correctly mapped to their generics, and the 1-0-1 schedule, SOS instruction, duration and follow-up were captured. The example also exposes current weaknesses: the tablet strengths 625 and 650 were not carried into the dose field, and a spurious Epinephrine entry was produced. Both questionable medicines were automatically flagged for review, and the rule engine highlighted the incomplete entries, so the clinician is directed to exactly the fields that need correction before confirmation. In a separate test, spoken numbers such as "six twenty five" were split into individual digits by the cleaner, indicating that compound-number normalization needs improvement.

F. Discussion and Limitations

The results indicate that the full voice-to-prescription workflow can run on CPU-only hardware without cloud services, preserving patient privacy and working in low-connectivity clinics. The layered design ensures graceful degradation: if the LLM is unavailable, the deterministic draft is still produced, and every output must be confirmed by the doctor. The present evaluation, however, begins from text transcripts, so it does not include acoustic noise, accents, code-switching or ASR word error rate; it measures recall but not precision, dose or frequency accuracy; and the cases were not independently adjudicated by clinicians. Speaker diarization, remote synchronization, authentication and encryption at rest are not yet implemented. The system is therefore a decision-support drafting tool and not an autonomous prescriber.

G. Privacy, Safety and Comparison

Because inference is local, the system avoids transmitting audio to third-party servers, which simplifies compliance with data-protection obligations such as India's Digital Personal Data Protection Act, 2023. Local processing alone does not guarantee confidentiality, however: the SQLite database, temporary audio files and PDFs still contain sensitive data and must be protected with authentication, encryption at rest and retention policies before clinical use. On the safety side, three independent safeguards reduce the impact of ASR or LLM errors: formulary validation with confidence-based review flags, deterministic rule alerts, and the mandatory draft-to-confirmed transition. Table VII compares the proposed design with representative approaches from the literature.

TABLE VII. COMPARISON WITH REPRESENTATIVE APPROACHES

Approach	Offline	Edge HW	Drug norm.	Rx output
Whisper ASR only [1], [8]	Yes	Partly	No	No
WhisperX [9]	Yes	No (GPU)	No	No
LLM clinical extraction [12]	No	No	Partly	No
Ambient AI scribes [13]	No	No	Partly	Notes
Proposed Aegis Clinical	Yes	Yes (Pi 5 target)	Yes	Yes (PDF)

Unlike ASR-only pipelines, the proposed system produces a structured prescription; unlike cloud LLM extractors and commercial ambient scribes, it requires no network connection and runs on a single-board computer. The trade-off is lower model capacity, which the system compensates for with a curated formulary, phonetic matching and rule-based checks.

V. CONCLUSION

This paper presented Aegis Clinical, an offline, edge-deployed assistant that converts doctor dictation into a structured, reviewable prescription. By combining a lightweight DSP front-end, clinically prompted int8 Whisper ASR, deterministic extraction with phonetic medicine normalization, optional 4-bit local LLM refinement and a clinical rule engine, the system achieved 87.69 % medication recall and 96 % diagnosis detection on a 50-case benchmark with a mean refinement latency of 9.55 s, all without internet connectivity. Future work will include end-to-end evaluation on real, clinician-annotated audio with word error rate and entity-level precision/F1, speaker diarization to isolate the doctor's voice, multilingual and code-switched dictation support, a larger formulary with a validated drug-interaction database, and security hardening with authentication, encryption and audit trails for clinical deployment.

REFERENCES

[1] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust speech recognition via large-scale weak supervision,” in Proc. 40th Int. Conf. Machine Learning (ICML), PMLR, vol. 202, 2023, pp. 28492–28518.
[2] A. Yang et al., “Qwen2.5 technical report,” arXiv preprint arXiv:2412.15115, 2024.
[3] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” arXiv preprint arXiv:2307.09288, 2023.
[4] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.
[5] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in Proc. Int. Conf. Learning Representations (ICLR), 2023.
[6] K. Singhal et al., “Large language models encode clinical knowledge,” Nature, vol. 620, pp. 172–180, 2023.
[7] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, “Large language models in medicine,” Nature Medicine, vol. 29, pp. 1930–1940, 2023.
[8] S. Gandhi, P. von Platen, and A. M. Rush, “Distil-Whisper: Robust knowledge distillation via large-scale pseudo labelling,” arXiv preprint arXiv:2311.00430, 2023.
[9] M. Bain, J. Huh, T. Han, and A. Zisserman, “WhisperX: Time-accurate speech transcription of long-form audio,” in Proc. Interspeech, 2023.
[10] A. Gulati et al., “Conformer: Convolution-augmented transformer for speech recognition,” in Proc. Interspeech, 2020.
[11] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.
[12] M. Agrawal, S. Hegselmann, H. Lang, Y. Kim, and D. Sontag, “Large language models are few-shot clinical information extractors,” in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2022.
[13] A. A. Tierney et al., “Ambient artificial intelligence scribes to alleviate the burden of clinical documentation,” NEJM Catalyst Innovations in Care Delivery, vol. 5, no. 3, 2024.
[14] D. Van Veen et al., “Adapted large language models can outperform medical experts in clinical text summarization,” Nature Medicine, vol. 30, pp. 1134–1142, 2024.
[15] A. Koenecke, A. S. G. Choi, K. X. Mei, H. Schellmann, and M. Sloane, “Careless Whisper: Speech-to-text hallucination harms,” in Proc. ACM Conf. Fairness, Accountability, and Transparency (FAccT), 2024.
[16] Y. Gu et al., “Domain-specific language model pretraining for biomedical natural language processing,” ACM Trans. Computing for Healthcare, vol. 3, no. 1, 2021.
[17] S. Henry, K. Buchan, M. Filannino, A. Stubbs, and O. Uzuner, “2018 n2c2 shared task on adverse drug events and medication extraction in electronic health records,” J. American Medical Informatics Association, vol. 27, no. 1, pp. 3–12, 2020.
[18] J. Chen and X. Ran, “Deep learning with edge computing: A review,” Proc. IEEE, vol. 107, no. 8, pp. 1655–1674, 2019.